

Lecture 10

Binary Search Trees



Slides are based on the textbook and its notes

Overview

□ *Search trees*

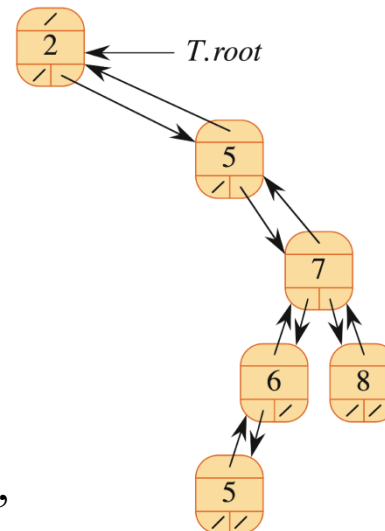
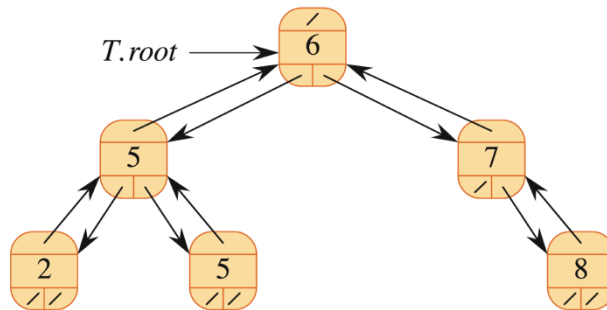
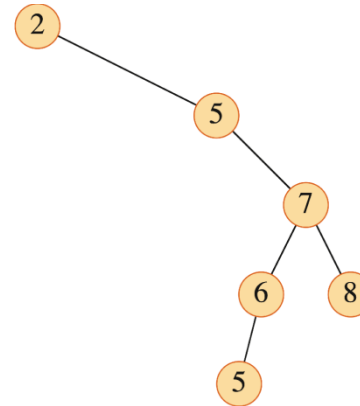
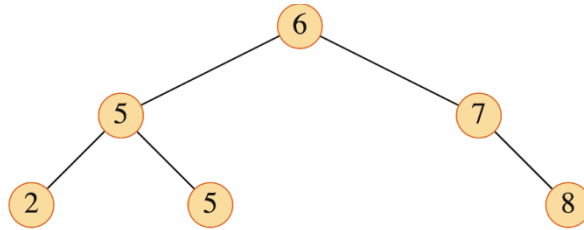
- Data structures that support many dynamic-set operations.
- Can be used as both a dictionary and as a priority queue.
- Basic operations take time proportional to the height of the tree.
 - For complete binary tree with n nodes: worst case: $\Theta(\lg n)$.
 - For linear chain of n nodes: worst case: $\Theta(n)$.
- Different types of search trees include binary search trees, red-black trees, and B-trees.

- We will cover binary search trees, tree walks, and operations on binary search trees.

Binary search trees

- ❑ Binary search trees are an important data structure for dynamic sets.
 - Accomplish many dynamics-set operations in $O(h)$ time, where h = height of tree.
 - As in Lecture 1/Section 10.3, represent a binary tree by a linked data structure in which each node is an object.
 - $T.root$ points to the root of the tree T .
 - Each node contains the attributes
 - ❑ *key* (and possibly other satellite data).
 - ❑ *left*: points to left child.
 - ❑ *right*: points to right child.
 - ❑ *p*: points to parent. $T.root.p = \text{NIL}$.
 - Store keys must satisfy the ***binary-search tree property***.
 - ❑ If y is in left subtree of x , then $y.key \leq x.key$.
 - ❑ If y is in right subtree of x , then $y.key \geq x.key$.

Binary search trees



(a) A binary search tree on 6 nodes with height 2.

(b) A less efficient binary search tree, with height 4, that contains the same keys.

(a)

(b)

Binary search trees

- ❑ Binary-search-tree property allows us to print keys in a binary search tree in order, recursively, using an algorithm called an *inorder tree walk*. Elements are printed in monotonically increasing order.
- ❑ How INORDER-TREE-WALK works:
 - Check to make sure that x is not NIL.
 - Recursively print the keys of the nodes in x 's left subtree.
 - Print x 's key.
 - Recursively print the keys of the nodes in x 's right subtree.

INORDER-TREE-WALK(x)

```
1  if  $x \neq \text{NIL}$ 
2      INORDER-TREE-WALK( $x.\text{left}$ )
3      print  $x.\text{key}$ 
4      INORDER-TREE-WALK( $x.\text{right}$ )
```

- ❑ **Time:** Intuitively, the walk takes $\Theta(n)$ time for a tree with n nodes, because it visits and prints each node once.

Binary search trees

- There are other algorithms that print the keys of a binary search tree:
 - PREORDER-TREE-WALK
 - Prints the root before the values in either subtree.
 - POSTORDER-TREE-WALK
 - Prints the root after the values in its subtrees.

Querying a binary search trees

□ *Searching*

TREE-SEARCH(x, k)

```
1  if  $x == \text{NIL}$  or  $k == x.\text{key}$ 
2      return  $x$ 
3  if  $k < x.\text{key}$ 
4      return TREE-SEARCH( $x.\text{left}, k$ )
5  else return TREE-SEARCH( $x.\text{right}, k$ )
```

Initial call is TREE-SEARCH($T.\text{root}, k$).

□ *Time*

The algorithm recurses, visiting nodes on a downward path from the root. Thus, running time is $O(h)$, where h is the height of the tree.

Querying a binary search trees

▣ *Searching: Iterative version*

The iterative version unrolls the recursion into a **while** loop. It is usually more efficient than the recursive version, since it avoids the overhead of recursive calls.

ITERATIVE-TREE-SEARCH(x, k)

```
1  while  $x \neq \text{NIL}$  and  $k \neq x.\text{key}$ 
2      if  $k < x.\text{key}$ 
3           $x = x.\text{left}$ 
4      else  $x = x.\text{right}$ 
5  return  $x$ 
```


Querying a binary search trees

□ *Minimum and maximum*

The binary-search-tree property guarantees that

- the minimum key of a binary search tree is located at the leftmost node, and
- the maximum key of a binary search tree is located at the rightmost node.

Traverse the appropriate pointers (*left* or *right*) until NIL is reached. In the following procedures, the parameter x is the root of a subtree. The first call has $x = T.root$.

TREE-MINIMUM(x)

```
1  while  $x.left \neq \text{NIL}$ 
2       $x = x.left$ 
3  return  $x$ 
```

TREE-MAXIMUM(x)

```
1  while  $x.right \neq \text{NIL}$ 
2       $x = x.right$ 
3  return  $x$ 
```

Time: Both procedures visit nodes that form a downward path from the root to a leaf. Both procedures run in $O(h)$ time, where h is the height of the tree.

Querying a binary search trees

□ *Successor and predecessor*

Assuming that all keys are distinct, the successor of a node x is the node y such that $y.key$ is the smallest key $> x.key$. We can find x 's successor based entirely on the tree structure. No key comparisons are necessary. If x has the largest key in the binary search tree, then x 's successor is NIL.

There are two cases:

1. If node x has a non-empty right subtree, then x 's successor is the leftmost node (minimum) in x 's right subtree.
2. If node x has an empty right subtree (x is the maximum in y 's left subtree), notice that:
 - As long as we move to the left up the tree (move up through right children), we are visiting smaller keys.
 - x has a successor y , then y is the lowest ancestor of x whose left child is also an ancestor of x .

Querying a binary search trees

TREE-SUCCESSOR(x)

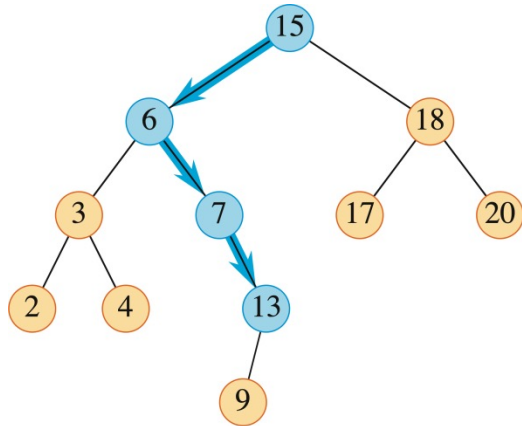
```
1  if  $x.right \neq \text{NIL}$ 
2      return TREE-MINIMUM( $x.right$ )  // leftmost node in right subtree
3  else // find the lowest ancestor of  $x$  whose left child is an ancestor of  $x$ 
4       $y = x.p$ 
5      while  $y \neq \text{NIL}$  and  $x == y.right$ 
6           $x = y$ 
7           $y = y.p$ 
8      return  $y$ 
```

□ TREE-PREDECESSOR is symmetric to TREE-SUCCESSOR.

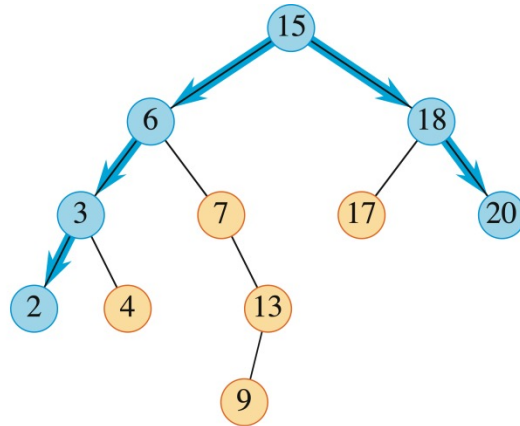
□ *Time*

Both the TREE-SUCCESSOR and TREE-PREDECESSOR procedures visit nodes on a path down the tree or up the tree. Thus, running time is $O(h)$, where h is the height of the tree.

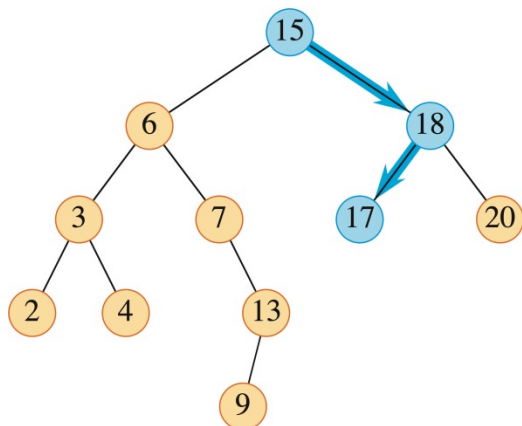
Querying a binary search trees



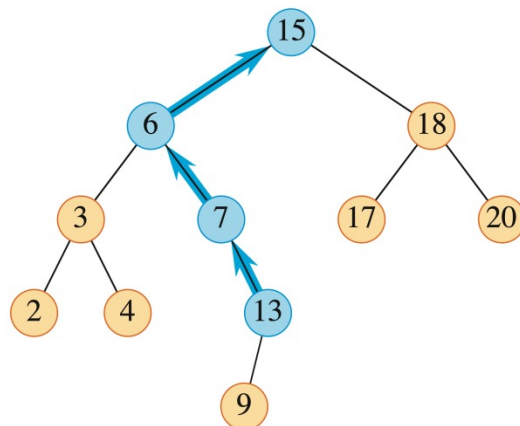
(a)



(b)



(c)



(d)

(a) A search for the key 13 in the tree follows the path 15 → 6 → 7 → 13 from the root.

(b) The minimum key in the tree is 2, which is found by following *left* pointers from the root. The maximum key 20 is found by following *right* pointers from the root.

(c) The successor of the node with key 15 is the node with key 17, since it is the minimum key in the right subtree of 15.

(d) The node with key 13 has no right subtree, and thus its successor is its lowest ancestor whose left child is also an ancestor. In this case, the node with key 15 is its successor.

Insertion and deletion

- ❑ Insertion and deletion allow the dynamic set represented by a binary search tree to change. The binary-search-tree property must hold after the change.
- ❑ **Insertion** is more straightforward than deletion.
 - To insert a new node into the binary search tree, the procedure is given node z , with $z.key$ already filled in, $z.left = \text{NIL}$, and $z.right = \text{NIL}$.
 - Beginning at root of the tree, trace a downward path, maintaining two pointers.
 - ❑ Pointer x : traces the downward path.
 - ❑ Pointer y : “*trailing pointer*” to keep track of parent of x .
 - Traverse the tree downward by comparing $x.key$ with $z.key$, and move to the left or right child accordingly.
 - When x is NIL , it is at the correct position for node z .
 - Compare $z.key$ with $y.key$, and insert z at either y 's *left* or *right*, appropriately.

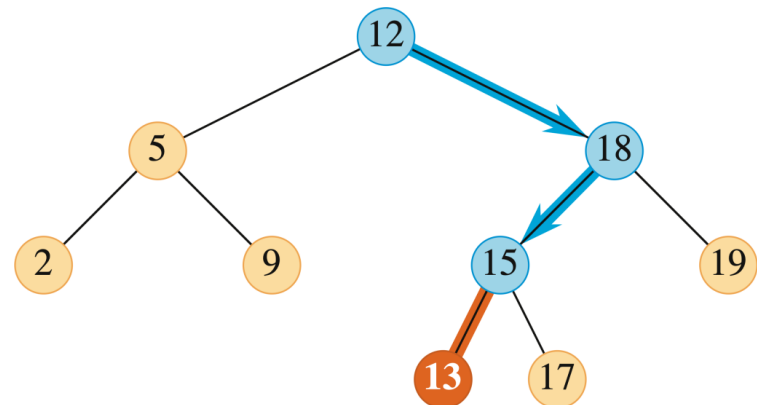
Insertion

TREE-INSERT(T, z)

```
1   $x = T.root$            // node being compared with  $z$ 
2   $y = NIL$               //  $y$  will be parent of  $z$ 
3  while  $x \neq NIL$       // descend until reaching a leaf
4       $y = x$ 
5      if  $z.key < x.key$ 
6           $x = x.left$ 
7      else  $x = x.right$ 
8   $z.p = y$              // found the location—insert  $z$  with parent  $y$ 
9  if  $y == NIL$ 
10      $T.root = z$       // tree  $T$  was empty
11 elseif  $z.key < y.key$ 
12      $y.left = z$ 
13 else  $y.right = z$ 
```

□ Time

Same as TREE-SEARCH. On a tree of height h , procedure takes $O(h)$ time.



Deletion

□ *Deletion*

Conceptually, deleting node z from binary search tree T has three cases:

1. If z has no children, just remove it by changing z 's parent to point to NIL instead of to z .
2. If z has just one child, then make that child take z 's position in the tree by changing z 's parent to point to z 's child instead of to z , dragging the child's subtree along.
3. If z has two children, then find z 's successor y and replace z by y in the tree. y must be in z 's right subtree and have no left child. The rest of z 's original right subtree becomes y 's new right subtree, and z 's left subtree becomes y 's new left subtree.

This case is a little tricky because the exact sequence of steps taken depends on whether y is z 's right child.

Deletion

- The code organizes the cases a bit differently.

Since it will move subtrees around within the binary search tree, it uses a subroutine, TRANSPLANT, to replace one subtree as the child of its parent by another subtree.

- TRANSPLANT(T, u, v) replaces the subtree rooted at u by the subtree rooted at v :

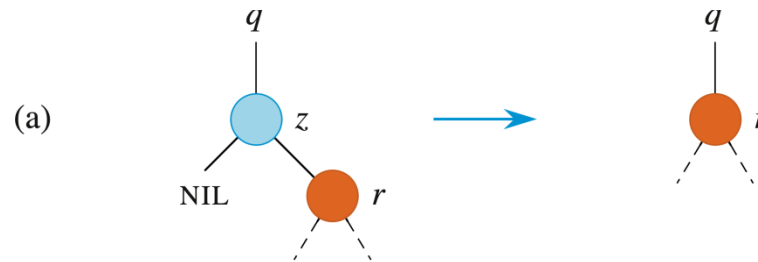
- Makes u 's parent become v 's parent (unless u is the root, in which case it makes v the root).
- u 's parent gets v as either its left or right child, depending on whether u was a left or right child.
- Doesn't update $v.left$ or $v.right$, leaving that up to TRANSPLANT's caller.

TRANSPLANT(T, u, v)

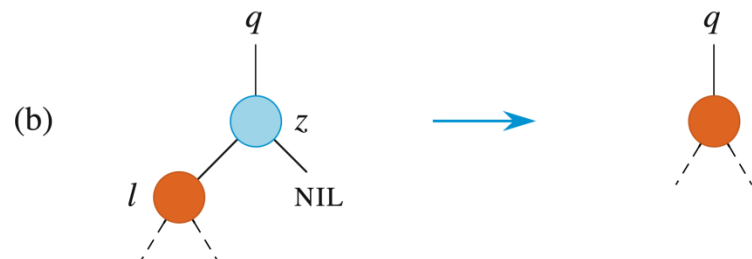
```
1  if  $u.p == \text{NIL}$ 
2       $T.root = v$ 
3  elseif  $u == u.p.left$ 
4       $u.p.left = v$ 
5  else  $u.p.right = v$ 
6  if  $v \neq \text{NIL}$ 
7       $v.p = u.p$ 
```


Deletion

- TREE-DELETE(T, z) has four cases when deleting node z from binary search tree T :
 - If node z has no left child, replace z by its right child r . The right child may or may not be NIL. (If node z 's right child is NIL, then this case handles the situation in which z has no children.)

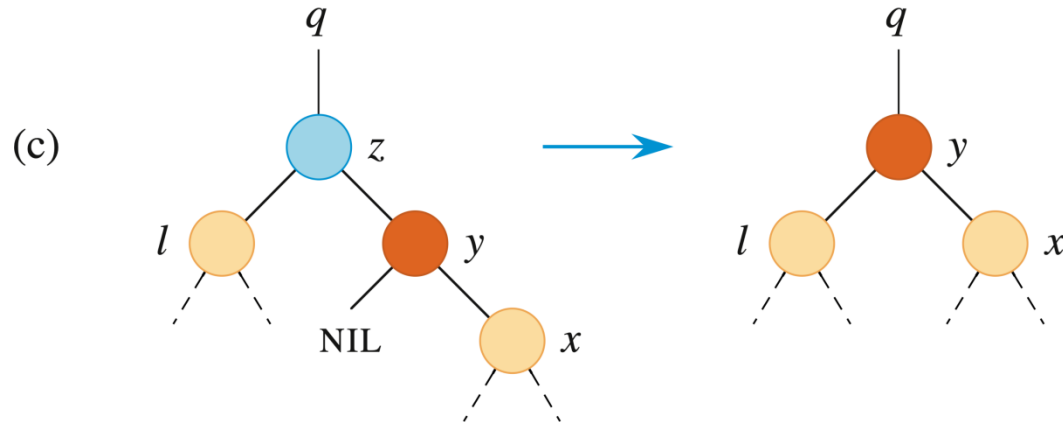


- If node z has a left child l but no right child, then replace z by its left child l .



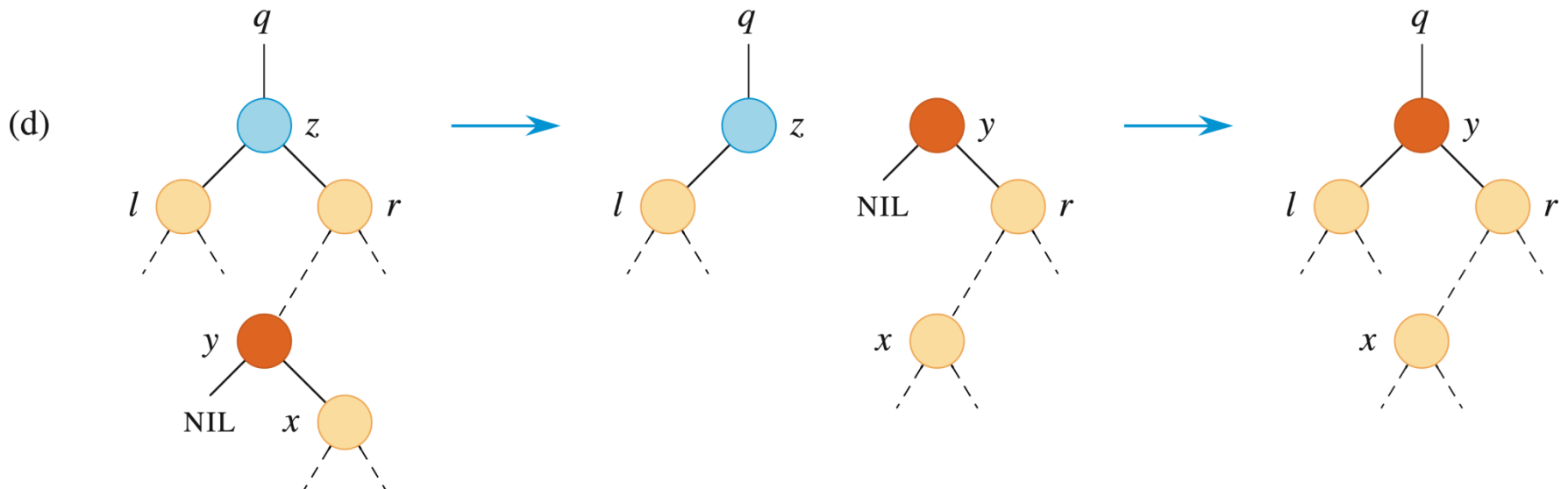
Deletion

- Otherwise, node z has two children. Find z 's successor y . Node y must lie in z 's right subtree and have no left child.
- Goal is to replace z by y , splicing y out of its current location.
 - If y is z 's right child, replace z by y and update y 's left child to become l , but leaving x as y 's right child.



Deletion

- Otherwise, y lies within z 's right subtree but is not z 's right child. First replace y by its own right child x and set y to be r 's parent. Then replace z by y .



Deletion

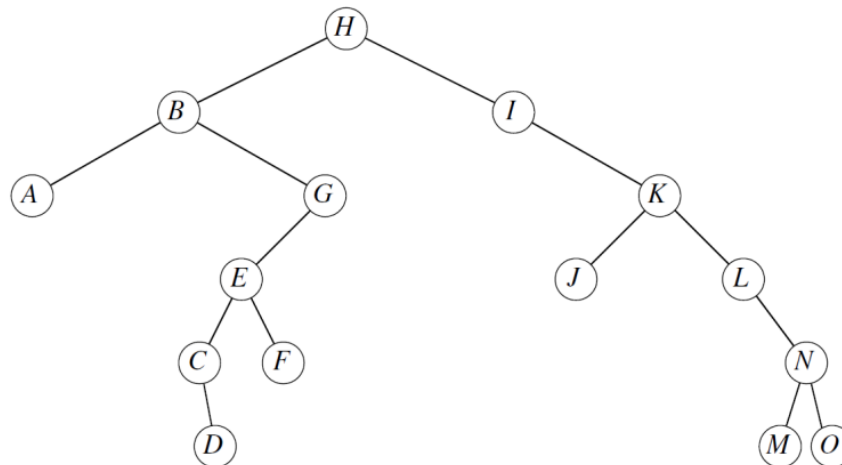
TREE-DELETE(T, z)

```
1  if  $z.left == \text{NIL}$ 
2      TRANSPLANT( $T, z, z.right$ )           // replace  $z$  by its right child
3  elseif  $z.right == \text{NIL}$ 
4      TRANSPLANT( $T, z, z.left$ )             // replace  $z$  by its left child
5  else  $y = \text{TREE-MINIMUM}(z.right)$         //  $y$  is  $z$ 's successor
6      if  $y \neq z.right$                      // is  $y$  farther down the tree?
7          TRANSPLANT( $T, y, y.right$ )        // replace  $y$  by its right child
8           $y.right = z.right$                 //  $z$ 's right child becomes
9           $y.right.p = y$                     //  $y$ 's right child
10     TRANSPLANT( $T, z, y$ )                  // replace  $z$  by its successor  $y$ 
11      $y.left = z.left$                       // and give  $z$ 's left child to  $y$ ,
12      $y.left.p = y$                         // which had no left child
```

- Note that the last three lines execute when z has two children, regardless of whether y is z 's right child.
- **Time:** Each line of TREE-DELETE, including the calls to TRANSPLANT, takes constant time, except for the call to TREE-MINIMUM in line 5. Thus, TREE-DELETE runs in $O(h)$ time on a tree of height h .

Deletion

- **Example:** On this binary search tree T , run the following. (You can either start with the original tree each time or start with the result of the previous call.)
- TREE-DELETE(T, I) shows the case in which the node deleted has no left child.
 - TREE-DELETE(T, G) shows the case in which the node deleted has a left child, but no right child.
 - TREE-DELETE(T, K) shows the case in which the node deleted has both children and its successor is its right child.
 - TREE-DELETE(T, B) shows the case in which the node deleted has both children and its successor is not its right child.



Reading

- ▣ Sections 12.1 ~ 12.3